Problem 3: Royal Archive Access Verification

Resolution Refutation using First Order Predicate Logic and Prolog

1 Problem and Goal

The archive contains sealed historical records. The given rules describe when a person can be trusted. We need to prove that Mira is trusted.

Goal: $trusted(mira)$

2 Task 1: Knowledge Representation

2.1 Predicates, constants and variables

The predicates used in the knowledge base are:

Predicate	**Meaning**
$archivist(X)$	X is an archivist.
$verifies(X, R)$	X verifies record R.
$trusted(X)$	X is trusted.
$has_access(X, R)$	X has access to record R.
$permitted_to_inspect(X, R)$	X is permitted to inspect R.
$inspects(X, R)$	X inspects record R.
$finds_hidden_mark(X, R)$	X finds a hidden mark in R.
$stored_in_royal_archive(R)$	R is stored in the royal archive.
$sealed(R)$	R is sealed.

The constants are $mira$ and $scroll_7$. The variables are X, representing a person, and R, representing a record.

2.2 Facts and rules in First Order Logic

The statements in the question can be written as follows.

Rule 1: Every archivist who verifies a record is trusted.

$$\forall X \forall R \, [(archivist(X) \land verifies(X, R)) \rightarrow trusted(X)]$$

Rule 2: Every trusted person who has access to a sealed record is permitted to inspect it.

$$\forall X \forall R \, [(trusted(X) \land has_access(X, R) \land sealed(R)) \rightarrow permitted_to_inspect(X, R)]$$

Rule 3: Anyone who inspects a record and finds a hidden mark in it verifies that record.

$$\forall X \forall R \, [(inspects(X, R) \land finds_hidden_mark(X, R)) \rightarrow verifies(X, R)]$$

Rule 4: Every record stored in the royal archive is sealed.

$$\forall R\,[stored_in_royal_archive(R) \rightarrow sealed(R)]$$

The given facts are:

$$F_1 : stored_in_royal_archive(scroll_7)$$
$$F_2 : has_access(mira, scroll_7)$$
$$F_3 : inspects(mira, scroll_7)$$
$$F_4 : finds_hidden_mark(mira, scroll_7)$$
$$F_5 : archivist(mira)$$

3 Task 2: Negation of the Goal

In resolution refutation, we add the negation of the statement we want to prove. The goal is $trusted(mira)$, so the negated goal is:

$$\boxed{\neg trusted(mira)}$$

If this negated goal leads to a contradiction with the knowledge base, then the original goal is proved.

4 Task 3: Conversion to Clause Form

4.1 Removing implications

We use the equivalence

$$A \rightarrow B \equiv \neg A \vee B.$$

Applying it to each rule gives:

$$R_1 : \neg archivist(X) \vee \neg verifies(X, R) \vee trusted(X)$$
$$R_2 : \neg trusted(X) \vee \neg has_access(X, R) \vee \neg sealed(R)$$
$$\vee\, permitted_to_inspect(X, R)$$
$$R_3 : \neg inspects(X, R) \vee \neg finds_hidden_mark(X, R)$$
$$\vee\, verifies(X, R)$$
$$R_4 : \neg stored_in_royal_archive(R) \vee sealed(R)$$

4.2 Negation movement, standardization and Skolemization

After implication removal, every negation is already directly in front of a predicate. Therefore, the formulas are in negation normal form.

The variables in different rules are standardized apart. For example, each rule can use its own variable names:

$$R_1 : (X_1, R_1), \quad R_2 : (X_2, R_2), \quad R_3 : (X_3, R_3), \quad R_4 : R_4.$$

There are no existentially quantified variables in the rules or facts. Hence, no Skolem constant or Skolem function is required.

4.3 Final CNF clauses

The resulting clause set is:

$$
\begin{aligned}
C_1 &: \neg archivist(X) \lor \neg verifies(X, R) \lor trusted(X) \\
C_2 &: \neg trusted(X) \lor \neg has_access(X, R) \lor \neg sealed(R) \lor permitted_to_inspect(X, R) \\
C_3 &: \neg inspects(X, R) \lor \neg finds_hidden_mark(X, R) \lor verifies(X, R) \\
C_4 &: \neg stored_in_royal_archive(R) \lor sealed(R) \\
C_5 &: stored_in_royal_archive(scroll_7) \\
C_6 &: has_access(mira, scroll_7) \\
C_7 &: inspects(mira, scroll_7) \\
C_8 &: finds_hidden_mark(mira, scroll_7) \\
C_9 &: archivist(mira) \\
C_{10} &: \neg trusted(mira) \qquad \text{(negated goal)}
\end{aligned}
$$

Each fact is a unit clause. The universal variables in C_1 to C_4 are instantiated when a resolution step requires them.

5 Task 4: Prolog Implementation

The Prolog source file is named `royal_archive_access.pl`. It contains:

1. the facts and rules from the knowledge base;
2. the negated goal;
3. comments describing implication removal, negation movement, standardization and Skolemization;
4. the CNF clauses;
5. a resolution procedure that selects complementary literals, unifies them and forms the resolvent;
6. the proof routine that prints each resolution step.

The source file is submitted separately as required by the assignment.

6 Task 5: Resolution Refutation

Only the clauses needed to prove the goal are used below. Clauses C_2 and C_4, along with facts C_5 and C_6, are still part of the knowledge base, but they are not needed for this particular proof.

Step 1: Use the inspection fact

Instantiate C_3 with $X = mira$ and $R = scroll_7$:

$\neg inspects(mira, scroll_7) \lor \neg finds_hidden_mark(mira, scroll_7) \lor verifies(mira, scroll_7)$.

Resolve this clause with C_7, which is $inspects(mira, scroll_7)$. The complementary literals $inspects$ and $\neg inspects$ cancel, leaving:

$$\boxed{R_1 : \neg finds_hidden_mark(mira, scroll_7) \vee verifies(mira, scroll_7)}$$

Step 2: Use the hidden mark fact

Resolve R_1 with C_8:

$$finds_hidden_mark(mira, scroll_7).$$

The complementary hidden mark literals cancel. The result is:

$$\boxed{R_2 : verifies(mira, scroll_7)}$$

This establishes that Mira verifies scroll 7.

Step 3: Use the archivist fact

Instantiate C_1 with $X = mira$ and $R = scroll_7$:

$$\neg archivist(mira) \vee \neg verifies(mira, scroll_7) \vee trusted(mira).$$

Resolve this clause with C_9, namely $archivist(mira)$. The result is:

$$\boxed{R_3 : \neg verifies(mira, scroll_7) \vee trusted(mira)}$$

Step 4: Derive that Mira is trusted

Resolve R_3 with R_2, which states $verifies(mira, scroll_7)$. The complementary verification literals cancel:

$$\boxed{R_4 : trusted(mira)}$$

Step 5: Derive the empty clause

Resolve R_4 with the negated goal C_{10}:

$$trusted(mira), \qquad \neg trusted(mira).$$

These are complementary unit clauses, so they resolve to:

$$\boxed{R_5 : []}$$

The empty clause represents a contradiction. Thus, the knowledge base together with the negated goal is inconsistent. The negated goal must therefore be false, and the original goal follows.

7 Task 6: Proof Verification

Run the Prolog program from the folder containing `royal_archive_access.pl`:

```
swipl -q -s royal_archive_access.pl -g prove_mira -t halt
```

The program should print the five resolution steps and finish with:

```
SUCCESS: Empty clause [] derived.
Therefore the negated goal is inconsistent with the knowledge base.
Hence trusted(mira) is proved.
```

The supplied execution screenshot is included in the `screenshots` folder of the submission ZIP.

8 Conclusion

Mira inspects scroll 7 and finds a hidden mark in it. By Rule 3, she therefore verifies the record. Since Mira is also an archivist, Rule 1 allows us to conclude that she is trusted. In the resolution refutation, adding the negation of this conclusion produces the empty clause. Therefore,

$$\boxed{trusted(mira)}$$

is proved.